```
print decbin(PHP_INT_MIN) . PHP_EOL;
print decbin(PHP_INT_MIN + 1) . PHP_EOL;
print decbin(PHP_INT_MIN + 2) . PHP_EOL;
print decbin(PHP_INT_MIN + 3) . PHP_EOL;
```

```
Output:
1000000000000000000000000000000000000000000000000000000000000000
1000000000000000000000000000000000000000000000000000000000000001
1000000000000000000000000000000000000000000000000000000000000010
1000000000000000000000000000000000000000000000000000000000000011
```

Hopefully this shows clearly how, as we start counting back toward zero from -2^{63}, we accumulate bits on the least significant (rightmost) side. This is why PHP is casting integers to floats when they exceed PHP_INT_MAX. Loss of precision can be a pain if not properly understood and managed, but integer overflow is by comparison catastrophic to calculations. It is relatively easy to break calculations, like this, for example:

```
$largeNum = PHP_INT_MAX + 1;
dumpLargeNum((int) $largeNum);

function dumpLargeNum(int $num): void
{
    var_dump($num);
}
```

```
Output:
int(-9223372036854775808)
```

The corollary to counting beyond PHP_INT_MAX is, of course, counting in reverse beyond PHP_INT_MIN, and logically the same problem integer overflow problem exists:

```
$largeNum = PHP_INT_MIN - 2000; // because loss of precision with float
dumpLargeNum((int) $largeNum);

function dumpLargeNum(int $num): void
{
    var_dump($num);
}
```